

Less transient constexpr allocation

Document #: P3032R1
Date: 2024-03-21
Project: Programming Language C++
Audience: EWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

Contents

- [1 Revision History](#)
- [2 Introduction](#)
 - [2.1 Immediate-escalating expressions](#)
 - [2.2 Transient allocations](#)
- [3 Proposal](#)
 - [3.1 Wording](#)
 - [3.2 Feature-Test Macro](#)
- [4 Acknowledgements](#)
- [5 References](#)

§ 1 Revision History

Since [\[P3032R0\]](#), fixed wording, added feature-test macro.

§ 2 Introduction

C++20 introduced constexpr allocation, but in a limited form: any allocation must be deallocated during that constant evaluation.

The intent of the rule is that no constexpr allocation persists to runtime. For more on why we currently need to avoid that, see Jeff Snyder's [\[P1974R0\]](#) and also [\[P2670R1\]](#).

But the rule cited above does slightly more than prevent constexpr allocation to persist until runtime. The goal of this paper is to allow more examples of allocations that *do not* persist until runtime, that nevertheless are still rejected by the C++23 rules.

For the purposes of this paper, we'll consider the example of wanting to get the number of enumerators of a given enumeration. While the specific example is using reflection ([\[P2996R1\]](#)), there isn't anything particularly reflection-specific about the example - it just makes for a good example. All you need to know about reflection to understand the example is that `^E` gives you an object of type `std::meta::info` and that this function exists:

```
namespace std::meta {
    using info = /* ... */;
    consteval vector<info> enumerators_of(info);
}
```

With that, let's go through several attempts at trying to get the *number* of enumerators of a given enumeration E as a constant:

	Attempt	Result
1	<pre>int main() { constexpr int r1 = enumerators_of(^E).size(); return r1; }</pre>	✅. This one is valid - because r1 is a <code>constexpr</code> variable, it's initializer starts a constant evaluation - which includes the entire expression. The temporary vector is destroyed at the end of that expression, so it doesn't persist outside of any constant evaluation.
2	<pre>constexpr int f2() { return enumerators_of(^E).size(); } int main() { constexpr int r2 = f2(); return r2; }</pre>	❌. This one is invalid. The same idea about initializing r2 as mentioned in the previous example is valid - but because f2 is a <code>constexpr</code> function, the invocation of <code>enumerators_of(^E)</code> is not in an immediate function context - so it needs to be a constant expression on its own. That is, we start a new constant evaluation within the original one - but this constant evaluation is just the expression <code>enumerators_of(^E)</code>. It is not the full expression <code>enumerators_of(^E).size()</code>. As a result, the temporary vector returned by <code>enumerators_of(^E)</code> persists outside of its constant expression in order to invoke <code>.size()</code> on it, which is not allowed.
3	<pre>consteval int f3() { return enumerators_of(^E).size(); } int main() { constexpr int r3 = f3(); return r3; }</pre>	✅. Both this row and the next row are subtle refinements of the second row that make it valid. The only difference between this and the previous row is that f2 was <code>constexpr</code> but f3 is <code>consteval</code>. This distinction matters, because now <code>enumerators_of(^E)</code> is no longer an immediate invocation - it is now in an immediate function context. As a result, the only thing that matters is that the entire

		expression <code>enumerators_of(^E).size()</code> is constant - and the temporary vector<info> does not persist past that.
4	<pre> template<class E> constexpr int f4() { return enumerators_of(^E).size(); } int main() { constexpr int r4 = f4<E>(); return r4; } </pre>	✓. Here <code>f4</code> is a <code>constexpr</code> function <i>template</i>, whereas <code>f2</code> was a regular <code>constexpr</code> function. This matters because of [P2564R3] - the fact that <code>enumerators_of(^E).size()</code> isn't a constant expression now causes <code>f4</code> to become a <code>consteval</code> function template - and thus we're in the same state that we were in <code>f3</code>: it's not <code>enumerators_of(^E)</code> that needs to be a core constant expression but rather all of <code>enumerators_of(^E).size()</code>.
5	<pre> consteval int f5() { constexpr auto es = enumerators_of(^E); return es.size(); } int main() { constexpr int r5 = f(); return r5; } </pre>	✗. Even though <code>f5</code> is <code>consteval</code>, we are still explicitly starting a new constant evaluation within <code>f5</code> by declaring <code>es</code> to be <code>constexpr</code>. That allocation persists past that declaration - <i>even though</i> it does not persist past <code>f5</code>, which by being <code>consteval</code> means that it does not persist until runtime.

Three of these rows are valid C++23 programs (modulo the fact that they're using reflection), but 2 and 5 are invalid - albeit for different reasons:

- in the 2nd row, we are required to consider `enumerators_of(^E)` as a constant expression all by itself - even if `enumerators_of(^E).size()` is definitely a constant expression.
- in the 5th row, we are required to consider `es` as a non-transient `constexpr` allocation - even though it definitely does not persist until runtime, and thus does not actually cause any of the problems that non-transient `constexpr` allocation has to address.

§ 2.1 Immediate-escalating expressions

The wording in [P2564R3] introduced the term *immediate-escalating expression* in 7.7 [expr.const]:

- 17 An expression or conversion is *immediate-escalating* if it is not initially in an immediate function context and it is either
- (17.1) — a potentially-evaluated *id-expression* that denotes an immediate function that is not a subexpression of an immediate invocation, or
 - (17.2) — an immediate invocation that is not a constant expression and is not a subexpression of an immediate invocation.

In the second example:

```
constexpr int f2() {  
    return enumerators_of(^E).size();  
}
```

The expression `enumerators_of(^E)` is immediate-escalating - it is an immediate invocation (`enumerators_of` is a `constexpr` function) that is not a constant expression (because the temporary vector persists outside of this expression). This is what causes `f2` to become a `constexpr` function template.

But `enumerators_of(^E).size()` is not an immediate invocation (it simply has a subexpression that is an immediate invocation). However, if we were to define it as an immediate invocation - then it would not be an immediate-escalating expression anymore because it is actually a constant expression. And that would be enough to fix this example (as well as `f4` which would then itself not escalate to `constexpr` since it wouldn't need to).

Put differently: instead of escalating `enumerators_of(^E)` up to the nearest function, which we then try to make `constexpr` (and fail in the case of `f2` because `constexpr` functions are not *immediate-escalating*), we only need to escalate up to the nearest enclosing expression that could be a constant expression.

§ 2.2 Transient allocations

The wording in 7.7 [\[expr.const\]](#) for rejecting non-transient allocations rejects an expression `E` as being a core constant expressions if `E` evaluates:

- (5.18) — a *new-expression* (`[expr.new]`), unless the selected allocation function is a replaceable global allocation function (`[new.delete.single]`, `[new.delete.array]`) and the allocated storage is deallocated within the evaluation of `E`;
- (5.20) — a call to an instance of `std::allocator<T>::allocate` (`[allocator.members]`), unless the allocated storage is deallocated within the evaluation of `E`;

That is - an allocation within `E` has to be transient to `E`. However, the rule we really want is that a constant allocation is transient to constant evaluation. In the fifth example:

```
constexpr int f5() {  
    constexpr auto es = enumerators_of(^E);  
    return es.size();  
}
```

The allocation in `enumerators_of(^E)` isn't transient to that expression, but it is definitely destroyed within `f5`, which is `constexpr`. That's important: if `f5` were `constexpr`, we'd have access to that allocation at runtime.

We can loosen the restriction such that an allocation within `E` must be deallocated within `E` or, if `E` is in an immediate function context, the end of that context. This would be the end of the `if constexpr { }` block or the end of the `constexpr` function. Such a loosening would allow `f5` above, but not if it's `constexpr`, and not if `es` were also declared `static`.

§ 3 Proposal

There are two separate potential changes here, that would each make one of the attempts above well-formed:

1. we could [escalate](#) expressions to larger expressions, so that `enumerators_of(^E).size()` becomes a constant expression, or
2. we could extend the notation of [transient allocation](#) to include the full immediate context instead of just the constant evaluation

The second of these is straightforward to word and provides a lot of value - since now particularly in the context of reflection you can declare a `constexpr vector<info>` inside a `consteval` function and use those contents as a constant expression. The first of these is complicated to word and does not provide as much value, as it is a limitation that is fairly easy to work around: either declare a local `constexpr` variable, or change the function to be `consteval` or a template.

As such, this paper only proposes extending the notion of transience.

§ 3.1 Wording

Change 7.7 [\[expr.const\]/5](#):

- ⁵ An expression E is a core constant expression unless the evaluation of E, following the rules of the abstract machine (`[intro.execution]`), would evaluate one of the following:

- (5.1) — [...]
- (5.18) — a *new-expression* (`[expr.new]`), unless the selected allocation function is a replaceable global allocation function (`[new.delete.single]`, `[new.delete.array]`) and the allocated storage is deallocated **either** within the evaluation of E **or, if E is in an immediate function context, within that context**;
- (5.19) — a *delete-expression* (`[expr.delete]`), unless it deallocates a region of storage allocated **either** within the evaluation of E **or, if E is in an immediate function context, within that context**;
- (5.20) — a call to an instance of `std::allocator<T>::allocate` (`[allocator.members]`), unless the allocated storage is deallocated **either** within the evaluation of E **or, if E is in an immediate function context, within that context**;
- (5.21) — a call to an instance of `std::allocator<T>::deallocate` (`[allocator.members]`), unless it deallocates a region of storage allocated **either** within the evaluation of E **or, if E is in an immediate function context, within that context**;
- (5.22) — [...]

§ 3.2 Feature-Test Macro

Bump the value of `__cpp_constexpr` in 15.11 [\[cpp.predefined\]](#):

```
- __cpp_constexpr 202306L  
+ __cpp_constexpr 2024XXL
```

§ 4 Acknowledgements

Thank you to Peter Dimov for being Peter Dimov and coming up with all of these examples.

§ 5 References

[P1974R0] Jeff Snyder, Louis Dionne, Daveed Vandevor. 2020-05-15. Non-transient constexpr allocation using `propconst`.

<https://wg21.link/p1974r0>

[P2564R3] Barry Revzin. 2022-11-11. `constexpr` needs to propagate up.

<https://wg21.link/p2564r3>

[P2670R1] Barry Revzin. 2023-02-03. Non-transient constexpr allocation.

<https://wg21.link/p2670r1>

[P2996R1] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevor. 2023-12-18. Reflection for C++26.

<https://wg21.link/p2996r1>

[P3032R0] Barry Revzin. 2024-02-13. Less transient constexpr allocation.

<https://wg21.link/p3032r0>